ThePremArchive — Premier League

Documentación de cada parte:

- Arañador (portada, necesidad de información, políticas, estadísticas, URLs semilla): `documentacion/01_crawler/documentacion_crawler.md`.
- Indexador (depuración de la colección, políticas de normalización, formato del índice, evaluación y estadísticas): `documentacion/02_indexador/documentacion_indexador.md`.

Este archivo es solo instrucciones técnicas para correr el código.

Estructura

- `common/` — módulos compartidos por ambos arañadores: políticas (`config.py`, `urls.py`, `extract.py`, `dedup.py`) y almacenamiento (`store.py`, `bitacora.py`). Define qué se descarga y por qué.
- `vanilla_spider_premierleague/` — arañador propio en Python (hilos, cola propia, descarga con `requests`). Define cómo se arma el motor de arañado.
- `scrapy_premierleague/` — arañador con Scrapy. Reusa `common/` tal cual; solo cambia el motor de arañado.
- `indexacion/` — indexación, con dos indexadores independientes que leen las mismas políticas: `indexador_manual/` (Python, archivo binario propio) e `indexador_lucene/` (Java, Apache Lucene 9.12). `config/` tiene las seis políticas de normalización (`politica_A..F.conf`) y las listas de datos que usan (`datos/`: stopwords, lemas, sinónimos, entidades, reglas de depuración, consultas de prueba). `herramientas/` tiene los scripts que construyen varias políticas seguidas, generan las listas y las tablas del documento, comparan la normalización de los dos indexadores y evalúan la calidad.

Instalación

```
pip install -r requirements.txt
```

El indexador de Lucene necesita además Java 17 o superior y Maven (ver su sección).

Arañador propio (Python desde cero, hilos)

Correr **desde la raíz del repo**:

```
python -m vanilla_spider_premierleague crawl --target-gb 10 -w 48 --delay 1 --per-
domain 3 -d 6
python -m vanilla_spider_premierleague status   # progreso y estadísticas
python -m vanilla_spider_premierleague reset    # tras un corte, devuelve a la
cola las URLs en vuelo
```

Detalle de cada política y dónde vive en el código: ver `documentacion_crawler.md`.

Arañador con biblioteca (Scrapy)

Correr desde cualquier carpeta (las rutas de salida están ancladas a la raíz del repo, no a la CWD):

```
cd scrapy_premierleague
scrapy crawl premierleague
```

Para una prueba corta antes de dejarlo corriendo horas:

```
scrapy crawl premierleague -s CLOSESPIDER_TIMEOUT=60 -s CLOSESPIDER_ITEMCOUNT=100
```

La diferencia real entre ambas implementaciones es el motor de arañado (hilos propios vs. el scheduler asíncrono de Scrapy) — la lógica de qué descargar, filtrar y deduplicar es literalmente el mismo código (`common/`).

Repositorio compartido

Ambos arañadores escriben al mismo lugar, sin importar cuál corras:

- `repo/` — el repositorio de texto plano final (`.txt` por documento, nombrado por hash de contenido)
- `data/crawl.db` — SQLite con metadatos y el índice de deduplicación
- `logs/bitacora.jsonl` (propio) y `logs/bitacora_scrapy.jsonl` (Scrapy) — bitácora de cada URL visitada

Como el nombre de archivo es el hash del contenido, si los dos arañadores bajan la misma noticia, no se duplica en el repositorio — y el pipeline de Scrapy consulta la misma base de datos para descartar duplicados y casi-duplicados contra lo que ya bajó el arañador propio (y viceversa).

```
python -m vanilla_spider_premierleague status   # sirve para ver el total
combinado de ambos
```

Indexador manual (Python, archivo binario propio)

Programa aparte de la araña: lee `repo/` (y `data/crawl.db` para depurar la colección) y escribe un índice TF-IDF binario en `indices/`. Correr **desde la raíz del repo**; cada política (`indexacion/config/politica_A..F.conf`) genera un índice:

```
python -m indexacion.indexador_manual build --config
indexacion/config/politica_A.conf
python -m indexacion.indexador_manual build --config
indexacion/config/politica_A.conf --limit 3000   # muestra para probar
python -m indexacion.indexador_manual info indices/manual_A.idx
python -m indexacion.indexador_manual buscar --config
indexacion/config/politica_A.conf -k 10 manchester united goals
```

```
hexdump -C indices/manual_A.idx | head -20      # Linux/macOS; en Git Bash de
Windows usar: xxd indices/manual_A.idx | head -20
```

El formato del archivo está descrito en `indexacion/indexador_manual/formato.py` y en la sección 4 del documento.

Con la colección completa depurada (1 010 827 documentos) cada política tarda entre 5 y 12 minutos y el índice pesa unos 650 a 800 MB. Además del `.idx` se escribe `indices/manual_X.idx.stats.json` con tiempos y tamaños. La depuración (qué documentos entran) se configura en `indexacion/config/datos/depuracion.txt`; `experimento_sin_depuracion.conf` y `experimento_filtro_tematico.conf` son experimentos que no forman parte de la entrega.

Si se interrumpe (Ctrl+C, corte de luz, cierre del equipo), no se pierde el trabajo: lo ya hecho queda en `indices/manual_X.idx.tmp/` y basta con volver a ejecutar **el mismo comando** para continuar donde se cortó. El `.idx` final solo aparece cuando está completo. Si cambia algo que afecta al resultado (el `.conf`, las listas de `indexacion/config/datos/`, el código de normalización o los documentos), el avance se descarta solo y empieza de cero; también se puede forzar con `--desde-cero`. Para cambiar la cantidad de procesos al continuar basta con `-p N` (por ejemplo en un equipo con menos núcleos).

Indexador con Apache Lucene (Java)

Programa aparte de la araña y del indexador manual. Lee los mismos `.conf` y las mismas listas de `datos/`, y aplica la misma depuración, así que cada política indexa los mismos documentos y los normaliza igual en los dos. Necesita Java 17 y Maven. Compilar una vez (deja `target/indexador-lucene.jar` con todas las dependencias adentro):

```
cd indexacion/indexador_lucene && mvn -q package
```

Correr **desde la raíz del repo**:

```
java -jar indexacion/indexador_lucene/target/indexador-lucene.jar build --config
indexacion/config/politica_A.conf
java -jar indexacion/indexador_lucene/target/indexador-lucene.jar build --config
indexacion/config/politica_A.conf --limit 3000   # muestra para probar
java -jar indexacion/indexador_lucene/target/indexador-lucene.jar info
indices/lucene_A
java -jar indexacion/indexador_lucene/target/indexador-lucene.jar buscar --config
indexacion/config/politica_A.conf -k 10 manchester united goals
```

El índice queda en la carpeta `indices/lucene_X/` (Lucene guarda cada estructura en un archivo aparte; `info` los lista con su tamaño y qué guarda cada uno) y sus resultados en `indices/lucene_X.stats.json`. Una sección opcional `[lucene]` en el `.conf` permite cambiar `salida`, `hilos`, `ram_buffer_mb`, `compresion` (BEST_SPEED | BEST_COMPRESSION) y `fusionar` (true = un solo segmento al final); el indexador manual la ignora. A diferencia del manual, no se reanuda si se interrumpe: hay que volver a ejecutarlo y empieza de cero.

Herramientas (`indexacion/herramientas/`)

```
python indexacion/herramientas/construir_indices.py                 # las seis políticas con el indexador manual
python indexacion/herramientas/construir_indices.py B C             # solo algunas
python indexacion/herramientas/construir_indices.py --motor lucene   # con Lucene (compilarlo antes)
python indexacion/herramientas/construir_indices.py --motor ambos    # manual y Lucene de cada política
python indexacion/herramientas/construir_indices.py --simular        # muestra los comandos sin ejecutarlos
python indexacion/herramientas/comparar_normalizacion.py --config indexacion/config/politica_C.conf --muestra 500   # ¿Lucene normaliza igual que el manual?
python indexacion/herramientas/generar_listas.py     # regenera stopwords.txt y lemas.txt (necesita nltk.download de stopwords y wordnet)
python indexacion/herramientas/generar_tablas.py     # tablas de estadísticas del documento, a partir de indices/*.stats.json
python indexacion/herramientas/evaluar.py pool       # junta los mejores resultados de las seis políticas para las consultas de prueba
python indexacion/herramientas/evaluar.py resultados # acierto en los 10 primeros resultados, con las marcas de estadisticas/evaluacion/pool_juzgado.csv
```

`construir_indices.py` corre las políticas en fila y no en paralelo, porque cada una ya usa todos los núcleos. `comparar_normalizacion.py` toma una muestra de documentos, los normaliza con los dos indexadores y compara la lista de términos de cada uno, en orden; termina con código 0 solo si son idénticas.

La evaluación se hace marcando en `estadisticas/evaluacion/pool_juzgado.csv` (columna `relevante`: 1 o 0) los documentos que devolvieron las políticas, al repetir `pool`, las marcas ya hechas se conservan y solo quedan por juzgar los documentos nuevos.

Archivo de resultados de cada indexador

Cada indexador deja un archivo de resultados por índice en `indices/` (`manual_A.idx.stats.json`, `lucene_A.stats.json`…) con estos campos: `documentos`, `tokens_totales`, `terminos`, `postings`, `tamano_bytes` y `segundos_total`. Los dos usan los mismos nombres, y con ellos `generar_tablas.py` arma las tablas. El de Lucene agrega el tamaño por tipo de archivo (`tamano_por_extension`) y el tiempo separado en listado, indexación y fusión de segmentos.